

CSC 490 Senior Capstone Proposal

Readscape

A Personal Reading Intelligence System

Project Team:

Hiwete Teshale

Huda Abu-Khass

Jasmine Butts

Table of Contents

1. Project Definition

- Why (it is needed)
- What (is the goal of the project)
- How (how will it be achieved)

2. Project Requirements Analysis (from the users' perspective, what they need)

- Functional
- Usability
 - User interface
 - Performance
- System
 - Hardware
 - Software
 - Database
- Security

3. Project Specification

- Focus / Domain / Area
- Libraries / Frameworks / Development Environment
- Platform
- Genre

4. System Design - Jasmine

- Overall System Architecture
- Identify subsystems
 - Teamwork Division

1. Project Definition

Readscape is a reading app that our group came up with because we noticed that keeping track of books can get confusing really fast. A person might save one book in their Notes app, screenshot another one from TikTok, save a few somewhere else, and then completely forget about half of them. That was basically where the idea started for us. We wanted to create one place where people can keep all of their books together without having to remember where they saved everything. In Readscape, users can organize books into categories like Want to Read, Currently Reading, and Finished, while also keeping track of their progress as they read.

The app is meant to be useful for different types of readers. Someone who already reads a lot can use it to keep their books and goals organized, while someone who is trying to read more can use it as motivation. Users can record their reading progress, ratings, completion dates, and look back at their reading history.

One of the main features we want to focus on is personalized recommendations. Instead of Readscape just being another app where people make a list of books, we want the information they enter to actually be useful. The app can use things like the books someone finished, what they rated highly, and the types of books they usually read to help suggest what they might enjoy next. For example, someone might think mystery is their favorite genre, but their reading history could show that they actually finish and rate fantasy books more often. Statistics and analytics can help users notice things about their reading habits that they probably would not notice from a regular list.

We also wanted Readscape to have a social side. Reading is usually something people do on their own, but people still enjoy talking about books and sharing recommendations with friends. Users will be able to add friends, join book clubs, participate in challenges, and possibly use an accountability partner feature. Friends could also compete based on things like books finished, pages read, or reading progress. We also talked about having university communities where students from the same school could connect with other readers or join reading groups. We think this could make the app feel more like a community instead of everyone just having their own separate account.

Overall, Readscape is our idea for a reading app that does more than just keep a list of books. We want users to be able to organize what they read, understand their reading habits, discover new books, and connect with other readers in one place. The main reason we chose this project is because it solves a problem that is easy to relate to, while also giving us the chance to work with more challenging parts like recommendations, analytics, databases, APIs, and social features. Our goal is to build something that is useful, simple to understand, and something we would actually want to use ourselves.

2. Project Requirements Analysis (from the users' perspective, what they need)

The requirements for Readscape are designed around the needs of readers who want a centralized and interactive way to manage their reading activity. The application must allow users to organize their personal libraries, track reading progress, interact with other readers, view reading statistics, and receive personalized recommendations. The system should also provide a secure and responsive experience across desktop and mobile devices.

- **Functional**
 - Create an account using Google OAuth 2.0.
 - Create and edit profile.
 - Search for books and authors using the Google Books API.
 - Scan a book's ISBN/barcode to locate book information.
 - Add books to personal digital library.
 - Categorize books as Want to Read, Currently Reading, or Finished.
 - Update reading progress, including the user's current page.
 - Rate and review books they have read. View their reading history.
 - View personalized reading statistics, including books completed, pages read, average book length, and preferred genres.
 - Receive personalized book recommendations based on reading history and preferences.
 - Interact with an AI reading assistant to ask questions about books and authors.
 - Maintain persistent AI chat history so previous conversations can be revisited.
 - Add and manage friends or accountability partners.
 - View reading activities shared by friends according to each user's privacy settings.
 - Participate in reading challenges or "book battles" with other users.
 - Receive relevant notifications about social activity, challenges, and reading goals.

- Manage privacy settings to control what information is visible to other users.
- Usability
 - The application should provide clear feedback when actions are completed successfully or when an error occurs. Features should be organized so that users can quickly access their library, reading statistics, recommendations, social features, and AI assistant.
 - The application should remain accessible using readable text, descriptive labels, keyboard-accessible controls where applicable, and responsive layouts.
 - User Interface Requirements:
 - Frontend- React and JavaScript
 - User dashboard with clickable icons to other features such as the personal library and status pages.
 - Visuals used to display reading statistics and pull data from Google Books API.
 - Consistent and clear design across all pages.
 - Performance
 - Reasonable response times in simple actions such as updating reading progress.
 - Availability of Google Books API and AI model at all times regardless of pressure.
 - Database queries should be designed efficiently, and unnecessary requests to external APIs should be minimized where practical.
- System
 - React
 - Spring Boot
 - PostgreSQL
 - Google Books API
 - Google OAuth 2.0
 - Hardware
 - Computer
 - Phone
 - Tablet

- Software
 - React
 - JavaScript Backend Java
 - Spring Boot Security
 - Spring Security Authentication
 - Google OAuth 2.0
 - PostgreSQL
 - Neon
 - Google Books API
- Database
 - Readscape will use PostgreSQL, hosted through Neon, to persist in application data.
 - The database will store user accounts and profiles, personal library records, reading progress, ratings and reviews, reading history, friendships, social information, challenges, and AI conversation.
 - Book metadata retrieved from the Google Books API
 - Database relationships will be maintained using appropriate primary and foreign keys. Spring Data JPA will provide the object-relational mapping between Java entities and PostgreSQL.
- Security
 - Readscape will use Spring Security and Google OAuth 2.0 to provide authentication and authorization.
 - Google OAuth will allow users to authenticate without Readscape directly managing their Google passwords.

3. Project Specification

- Focus / Domain / Area: Personal reading management and reader engagement

Libraries / Frameworks / Development Environment

- Backend Framework – Spring Boot
- Language – Java
- Security – Spring Security
- Authentication – Google OAuth 2.0
- Database – PostgreSQL
- Database Tool - Neon
- ORM – Spring Data JPA
- Books API – Google Books API
- AI – LLM API + Java SDK/client
- Frontend – React / JavaScript

- API Communication – REST / JSON
 - Book Scanning – Google ML Kit Code Scanner API
 - API Testing – Postman or Bruno
 - Build Tool – Maven
 - Version Control – Git + GitHub
 - IDE – Visual Studio Code
 - Containerization – Docker
- Platform – Mobile and Web Application
 - Genre – AI-Powered Social Reading and Book Management Platform

4. System Design

- System Architecture

Readscape will utilize a three-tier client-server architecture with a React-based presentation layer, a Spring Boot application layer, and a PostgreSQL data layer. Backend communication will be facilitated by RESTful APIs that connect to the frontend, database, and external services. Spring Security and Google OAuth 2.0 will be used for authentication and authorization, while the AI subsystem will use user reading history, book metadata, and conversation history to provide personalized assistance and recommendations.
- Subsystems
 - User & Social Management
 - User authentication
 - User profiles
 - Google OAuth 2.0
 - Spring Security
 - Friends
 - Accountability partners
 - Notifications
 - Privacy settings
 - Book Management & Discovery
 - Personal library
 - Reading status
 - Reading progress
 - Ratings/reviews
 - Book scanner
 - ISBN/barcode recognition
 - Google Books API

- Book/author information
- Reading history
- Book search/discovery
- AI Reading Assistant
 - AI chatbot
 - Persistent chat history
 - Book Q&A
 - Author Q&A
 - Personalized recommendations
 - Reading-history analysis
 - Reading insights
 - Potentially AI-powered accountability/battle insights
- Teamwork Division
 - Hiwete: Backend Development, API Implementation, Database Integration and Management
 - Huda: Front End Development
 - Jasmine: AI Integration, Application Security & Data Protection